Cordon — User Stories & On-Chain Requirements

Janhavi Chavada · Turbin3 Builders Cohort

May 2026

Part A — User Stories & On-Chain Requirements

Project: Cordon — transaction firewall for Solana AI agents **Author:** Janhavi Chavada · Turbin3 Builders Cohort **Input document:** `cordon-proposal.pdf` (Capstone Assignment 1)

This document is the clean, final deliverable. The process — prompts, AI outputs, my critiques, refinement log — lives in `part-b-process-appendix.md`.

---

1. Refined value proposition (one-line recap)

Cordon sits between an AI agent's signing path and the Solana network. Every transaction the agent wants to broadcast first gets simulated, then policy-checked against an on-chain Anchor program, then (when flagged) queued for a human approver before it ever hits the network. Policies, the approver set, and the kill switch live on-chain so a single operator can't quietly raise a cap or remove a control.

The POC has to prove **one loop end-to-end**: an agent's intent is intercepted, checked against on-chain policy, and either auto-broadcast or routed to a human approver who decides. Everything in this document is scoped to that loop.

---

2. Core user personas (final, 4)

The brainstorm produced ~12 candidate user types (see appendix B.2). Four made it through prioritization. The rest are either downstream consumers of POC outputs (compliance reader, end user of an agent-managed treasury, protocol risk manager) or post-POC stakeholders (insurance underwriter, regulator). They are not in the critical path for proving the loop works.

P1 — Agent integrator

The developer who drops `cordon-rs` (or its TypeScript binding) into an agent codebase. Writes the initial policy, wires the agent's signing path through the SDK, points the dashboard at the right program ID. Primary buyer.

P2 — Agent signer

The programmatic actor running inside the integrator's agent — the runtime entity that builds and submits transaction intents through the SDK. **Not** the same as the integrator: the integrator writes the agent's code; the agent signer is the running process whose intents Cordon intercepts. Drawing this line matters because the integrator has policy-mutation rights and the agent signer does not.

P3 — Approver

A human (or, in production, a Squads multisig) listed in the policy's approver set. Watches the Next.js dashboard. Approves or rejects flagged intents.

P4 — Policy authority

The on-chain owner of the policy registry account for a given agent. Mutates spend caps, allowlists, time windows, the approver set, and the kill switch. Day-1 default is the integrator's wallet; production target is a multisig or DAO. Distinct from the approver: the policy authority changes the rules; the approver applies them on a single pending transaction.

Why these four and not more

The minimal cast that fully exercises the loop is: someone who configured it (P1 + P4), the actor whose intents get checked (P2), the human who decides on flagged ones (P3). Add a fifth and you're adding someone who reads outputs, not someone who drives the critical path. The cohort timeline doesn't have room for personas that don't shift the POC scope.

---

3. Function map

One concrete, single-action function per row. No "manages" or "configures" — those are categories, not actions.

P1 — Agent integrator

#	Function
F1.1	Installs the Cordon SDK in their agent project
F1.2	Generates the agent signer keypair the SDK will wrap
F1.3	Calls `register_agent` on-chain to create the policy account
F1.4	Reads the agent's pending-approval queue from the dashboard
F1.5	Reads the on-chain audit log for their agent

P2 — Agent signer

#	Function
F2.1	Builds an intended Solana transaction
F2.2	Submits the intent to Cordon for a policy check
F2.3	Receives an auto-approval and broadcasts the original transaction
F2.4	Receives a "queued for HITL" response and pauses

#	Function
F2.5	Receives a “policy denied” response and aborts
F2.6	Receives an approver’s decision on a previously queued intent

P3 — Approver

#	Function
F3.1	Connects their wallet to the dashboard
F3.2	Views the pending-approval queue for an agent they’re listed on
F3.3	Opens a pending intent and reads the simulation output plus the policy reason it was flagged
F3.4	Approves a pending intent
F3.5	Rejects a pending intent
F3.6	Views past decisions (their own and other approvers’)

P4 — Policy authority

#	Function
F4.1	Sets the per-asset spend cap on the policy account
F4.2	Adds a program ID to the CPI allowlist
F4.3	Removes a program ID from the CPI allowlist
F4.4	Adds an approver public key to the approver set
F4.5	Removes an approver public key from the approver set
F4.6	Flips the kill switch (pauses all of the agent’s intents)
F4.7	Transfers policy authority to a new wallet (e.g., a Squads multisig)

---

4. Top critical user interactions for the POC

Two interaction paths. They share most of the on-chain machinery, so building one gets you most of the other.

Path A — Auto-approval loop

The agent submits an intent. The on-chain policy check passes (under the spend cap, target program is on the allowlist, inside the time window, under the rate limit). The SDK broadcasts the original transaction. The decision is written to the on-chain log.

Path B — HITL loop

The agent submits an intent. The on-chain policy check fails one or more rules. A `PendingApproval` account is created. The agent's submit call returns "queued." The approver sees the queued intent in the dashboard, opens it, and either approves or rejects. On approval, the SDK broadcasts. On rejection or expiry, the intent is dropped. Either way, the decision is written to the on-chain log.

These two paths cover every persona, every Cordon-side instruction the POC needs, and every account type. If both work end-to-end on devnet, the POC is done.

---

5. Final user stories (atomic, de-jargoned)

Output of Part C (full refinement log in appendix B.7). Each story is a single action, in plain language, with a clear actor and outcome. Numbering is stable so the on-chain requirements in §6 can reference back.

Setup stories

- **S1.** As an integrator, I install the Cordon SDK in my agent's project.
- **S2.** As an integrator, I generate the keypair my agent will sign with.
- **S3.** As an integrator, I create my agent's on-chain policy account.
- **S4.** As a policy authority, I set the per-asset spend cap on the policy account.
- **S5.** As a policy authority, I add a target program to the allowlist.
- **S6.** As a policy authority, I add an approver's public key to the approver set.

Runtime stories — auto-approval path

- **S7.** As an agent signer, I build a transaction I want to broadcast.
- **S8.** As an agent signer, I submit that transaction to Cordon for a policy check.
- **S9.** As Cordon, I record the intent on-chain so the decision is auditable.
- **S10.** As Cordon, I check the intent against the policy account's rules.
- **S11.** As Cordon, I mark the intent approved when every rule passes.
- **S12.** As an agent signer, I broadcast the original transaction once Cordon marks the intent approved.

Runtime stories — HITL path

- **S13.** As Cordon, I mark the intent as pending when one or more rules fail.
- **S14.** As Cordon, I record the specific rule that triggered the pending state.
- **S15.** As an approver, I open the dashboard and see every pending intent on agents I'm listed on.
- **S16.** As an approver, I open a single pending intent and read its simulation output and the rule it tripped.
- **S17.** As an approver, I approve a pending intent.
- **S18.** As an approver, I reject a pending intent.
- **S19.** As an agent signer, I broadcast the original transaction once an approver marks the pending intent approved.
- **S20.** As an agent signer, I drop the transaction when an approver rejects it or the pending intent expires.

Governance stories

- **S21.** As a policy authority, I flip the kill switch to pause every new intent for my agent.
- **S22.** As a policy authority, I transfer policy authority to a new wallet.

Read stories

- **S23.** As an integrator, I read my agent's pending-approval queue.
- **S24.** As an integrator, I read my agent's on-chain decision log for a date range.

---

6. Potential on-chain requirements per story

Brainstorm output (Part D). Each story is followed by a bulleted list of on-chain needs. These are deliberately at the requirement level — not the implementation level — so the next assignment (smart-contract architecture) can decide between competing designs. "PDA" and "instruction" are used because Cordon is Anchor; that's not jargon at the requirements stage, it's the substrate.

Anchor program working name: `cordon_policy`. Two account types carry most of the state: `Policy` (one per agent, owns the rules and the approver set) and `Intent` (one per submitted transaction, owns its lifecycle).

Setup

S1 — Integrator installs the SDK - No on-chain requirements. Off-chain only (Cargo / npm).

S2 — Integrator generates the agent signer keypair - No on-chain requirements at this step. The keypair only matters once it signs.

S3 — Integrator creates the agent's policy account - Need a `register_agent` instruction. - It creates a `Policy` PDA with seeds `[b"policy", agent_signer_pubkey]`. - The `Policy` stores the policy authority's pubkey, the agent signer's pubkey, a `paused` flag (default `false`), a `bump`. - The `Policy` initializes empty containers for the allowlist, approver set, and rule fields. - The instruction fails if a `Policy` already exists for that agent signer. - The instruction emits a `PolicyCreated` event for the audit log.

S4 — Policy authority sets the spend cap - Need a `set_spend_cap` instruction. - It mutates `Policy.spend_cap_lamports` (per-asset cap — for the POC, just SOL; extend per-mint later). - It is gated on the signer matching `Policy.authority`. - It emits a `PolicyChanged` event with the old and new cap. - It fails if the `Policy` is paused (kill-switch state) so that paused policies can't be silently re-tuned.

S5 — Policy authority adds a program to the allowlist - Need an `allowlist_add` instruction. - It pushes a program ID onto `Policy.allowed_programs` (bounded vec, e.g. max 32 entries; size pre-allocated to avoid reallocation cost). - It is gated on `Policy.authority`. - It fails if the program ID is already on the list (idempotency / duplicate prevention). - It emits a `PolicyChanged` event.

S6 — Policy authority adds an approver - Need an `approver_add` instruction. - It pushes a pubkey onto `Policy.approvers` (bounded vec, e.g. max 8). - It is gated on `Policy.authority`. - It fails if the pubkey is already in the set. - It emits a `PolicyChanged` event.

Auto-approval runtime path

S7 — Agent signer builds a transaction - No on-chain requirements. The agent assembles the tx off-chain via the SDK.

S8 — Agent signer submits the intent to Cordon - Need a `submit_intent` instruction. - It creates an `Intent` PDA with seeds `[b"intent", agent_signer_pubkey, policy.nonce.to_le_bytes()]`, where `policy.nonce` is a monotonic counter on `Policy` (incremented atomically inside this instruction). - The `Intent` stores a hash of the original transaction message (not the full tx — keeps account size bounded), the target program IDs the tx CPIs into, the lamports moved, the timestamp, the submitter, and a `status` enum (`Submitted`, `AutoApproved`, `Pending`, `HumanApproved`, `Rejected`, `Expired`). - The signer must match `Policy.agent_signer`. - The instruction fails if `Policy.paused` is `true` (kill switch honored at submission, not just at check). - It emits an `IntentSubmitted` event.

S9 — Cordon records the intent on-chain - Covered by S8 — the `Intent` account *is* the on-chain record. The append-only audit trail is the set of `Intent` accounts plus the event stream; no separate "log" account needed.

S10 — Cordon checks the intent against the policy - Need a `check_intent` instruction (could be folded into `submit_intent` for the POC; kept separate here in case off-chain simulation needs to run between the two). - It reads the `Intent` and the matching `Policy`. - It evaluates: `lamports <= spend_cap`, every CPI target $\in$ `allowed_programs`, current slot/clock is inside the time window, and the per-rate-limit-period count is under the cap (rate-limit counter is stored on `Policy` with a sliding window or a simple "count + window start"). - It writes `Intent.status` based on the result (`AutoApproved` or `Pending`). - It writes `Intent.failure_reason` (a small enum: `OverCap`, `ProgramNotAllowed`, `OutsideTimeWindow`, `RateLimited`) when transitioning to `Pending`, so the dashboard can show *why* without recomputing.

S11 — Cordon marks the intent approved when every rule passes - Covered by S10's state transition to `AutoApproved`. - It emits an `IntentAutoApproved` event.

S12 — Agent signer broadcasts the approved transaction - No additional on-chain requirements for the broadcast itself (broadcast is just sending the tx to RPC). - Requires that the SDK can prove approval to itself before broadcasting; reading `Intent.status == AutoApproved` is sufficient. - Optional hardening for later: a separate `release` instruction that burns the `Intent` account and returns rent to the integrator, called after broadcast confirms.

HITL runtime path

S13 — Cordon marks intent as pending when a rule fails - Covered by S10's state transition to `Pending`. - It emits an `IntentPending` event with the rule that tripped it (the same data written to `Intent.failure_reason`).

S14 — Cordon records which rule was tripped - Covered by S10 via `Intent.failure_reason`.

S15 — Approver sees pending intents for agents they're on - No new on-chain instruction. The dashboard reads pending intents via `getProgramAccounts` with a memcmp filter on `Intent.status == Pending` plus a filter on `Intent.agent_signer ∈ {agents where this approver is in Policy.approvers}`. - Requires `Intent` account layout to be memcmp-friendly: the `status` byte at a fixed, low offset; `agent_signer` at a fixed offset.

S16 — Approver opens a single pending intent - No on-chain instruction. Read-only fetch of one `Intent` account, plus the matching `Policy` for context. - Requires `Intent.failure_reason` and the cached lamports / CPI target list to be on the account (so the dashboard doesn't need to recompute or re-fetch the original tx to explain the flag).

S17 — Approver approves a pending intent - Need an `approve_pending` instruction. - It is gated on the signer being in `Policy.approvers`. - It is gated on `Intent.status == Pending`. - It transitions `Intent.status` to `HumanApproved`. - It writes `Intent.approver` and `Intent.decided_at_slot`. - It fails if the intent has expired (`current_slot > Intent.submitted_at_slot + Policy.pending_ttl_slots`); in that case the instruction transitions `Intent.status` to `Expired` instead. - It emits an `IntentHumanApproved` event. - (For the POC, single-signature approval is enough. Multi-signature approval — m-of-n on `Policy.approvers` — is a clean extension: store an approval bitmap on `Intent`, count signatures, transition state when the threshold is met.)

S18 — Approver rejects a pending intent - Need a `reject_pending` instruction. - Same gating as S17 (signer in `Policy.approvers`, status `Pending`). - Transitions `Intent.status` to `Rejected`. - Writes `Intent.approver`, `Intent.decided_at_slot`, and optionally a small `Intent.reject_reason` enum. - Emits an `IntentRejected` event.

S19 — Agent signer broadcasts after human approval - Same as S12. SDK reads `Intent.status == HumanApproved` before broadcasting.

S20 — Agent signer drops the transaction on rejection or expiry - No on-chain instruction required. The SDK reads `Intent.status ∈ {Rejected, Expired}` and aborts. - Requires the expiry semantics from S17 to be enforced *on-chain* at the next `approve_pending` / `reject_pending` call, OR an explicit `expire_intent` instruction that anyone can call. The latter is cleaner because it doesn't require waiting for the next approver action to garbage-collect.

Governance

S21 — Policy authority flips the kill switch - Need a `set_paused` instruction. - It mutates `Policy.paused`. - It is gated on `Policy.authority`. - It emits a `PolicyChanged` event. - It affects `submit_intent` (S8 fails when paused) and `set_spend_cap` (S4 fails when paused, so a compromised authority can't pause-then-loosen). - Open question for the next assignment: should `approve_pending` also fail when paused? Argument for: kill switch is total. Argument against: an approver may want to reject queued intents while paused. Default to "approve fails, reject still works" — the kill switch should never block a *safer* action.

S22 — Policy authority transfers authority - Need a `transfer_authority` instruction. - It mutates `Policy.authority` to a new pubkey. - It is gated on the current `Policy.authority`. - Should be a two-step transfer (propose → accept) to avoid transferring authority to a pubkey nobody controls. Adds a `Policy.pending_authority` field. - Emits a `PolicyChanged` event.

Read stories

S23 — Integrator reads the pending queue - No on-chain instruction. `getProgramAccounts` with a memcmp filter on `status == Pending` and `agent_signer == <theirs>`. - Requires the account-layout property already noted in S15.

S24 — Integrator reads the decision log for a date range - No on-chain instruction. The decision log is the set of `Intent` accounts with a terminal status (`AutoApproved`, `HumanApproved`, `Rejected`, `Expired`). - Date-range filtering: `Intent.submitted_at_slot` is on the account; the SDK converts slot ranges to wall-clock via `getBlockTime`. - For the POC this is acceptable; for production, the volume of `Intent` accounts will require either (a) a `release` instruction that closes terminal intents after archiving them off-chain to an indexer, or (b) a compression scheme. Out of scope for this assignment, flagged for architecture.

---

7. Open questions handed to the next assignment

These are the things I deliberately didn't decide here, because they belong in the architecture assignment (smart contract design and diagramming), not in requirements.

1. **Should `check_intent` be a separate instruction or folded into `submit_intent`?** Trade-off: separating it lets off-chain simulation happen between the two, but doubles the round trips. Likely fold them together for the POC and split if simulation needs the gap.
2. **Account size for the bounded vecs (allowlist, approvers).** Picked illustrative maxes (32 programs, 8 approvers); the architecture assignment should pick real ones based on rent cost and realistic fleet shapes.
3. **Rate-limit window representation.** Sliding window vs. fixed window vs. token bucket — each has different on-chain math. Picking one is an architecture call.
4. **Intent garbage collection.** Either a `release` instruction or some kind of compression. Required before mainnet, not required for the devnet POC.

5. **Squads integration for the approver path.** The persona definition assumes an approver could be a Squads multisig, but how Cordon verifies a Squads approval signature inside `approve_pending` is an architecture question, not a requirements one.

Part B — Process Appendix

The clean deliverable lives in `part-a-deliverable.md`. This file is the log of how I got there: prompts I ran, what the AI gave back, what I agreed with, what I overrode, and the explicit before/after refinement log for Part C.

B.1 — Inputs

The single input to this assignment is the refined project proposal from the previous Capstone assignment (`cordon-proposal.pdf`, included at the root of this repo). The value proposition I'm carrying forward, verbatim from §1 of that proposal:

> *Cordon is the policy enforcement and HITL approval layer that sits between an AI agent's intent and the Solana network. You drop the Rust SDK (`cordon-rs`) or its TypeScript binding into any agent built on SendAI Agent Kit, Arc, or a custom Rig pipeline. You write your policies once into an Anchor program: per-asset spend caps, an allowlist of programs the agent can CPI into, time-of-day windows, rate limits, daily volume ceilings, anomaly thresholds. Anything the policy doesn't auto-approve gets pushed to a Next.js dashboard where authorized signers approve or reject before the agent broadcasts.*

Three target user *segments* came out of that assignment: agent dev teams (primary), hackathon teams (beachhead), and protocols / DAOs (expansion). This assignment narrows from segments to **personas inside one agent deployment**, which is a different cut.

B.2 — Manual brainstorm of user types (Part A.1)

Before prompting the AI, I wrote down every user type that could plausibly interact with a single Cordon deployment, by the four categories the assignment asks for.

Direct users

- **Agent integrator** — the developer installing the SDK into their agent codebase.
- **Agent signer** — the programmatic actor inside the agent that builds and submits transactions.
- **Approver** — the human reviewing flagged intents in the dashboard.
- **Policy author** — the person writing the policy rules. (Often the integrator on day 1.)
- **Policy authority** — the on-chain owner of the policy registry account (often a multisig).

Indirect users / beneficiaries

- **End user of an agent-managed treasury** — e.g. a DAO member whose funds the agent manages. Doesn't touch Cordon directly but benefits when a bad intent gets blocked.

- **Token holders of a DAO whose treasury is agent-managed** — same as above but at protocol level.
- **Protocol risk manager** — at Drift, Jupiter, Kamino, etc. — wants to know the agents integrating with their protocol have firewalls in front of them.

Administrators / moderators

- **Cordon platform operator (me)** — running the free hosted tier of the dashboard.
- **Compliance officer / auditor** — at an enterprise running Cordon, reads the on-chain audit log to verify controls.
- **Incident responder** — during a live security event, needs to pause an agent fast.

Stakeholders

- **Founder / engineering lead** at the agent dev company — owns the buying decision.
- **Insurance underwriter** — would underwrite agent operations differently with Cordon in front of them.
- **Regulator** — cares about the audit log existing, not about using Cordon directly.
- **Integration partners** — Squads (HITL signing target), SendAI (host for Cordon as a plugin).

Twelve user types total. This is the input to the prioritization step.

B.3 — AI prioritization prompt (Part A.2)

My prompt:

> *My project's value proposition is: Cordon is a policy enforcement and HITL approval layer for AI agents on Solana. The SDK intercepts every transaction the agent wants to broadcast, checks it against an on-chain Anchor policy program (spend caps, CPI allowlist, time windows, rate limits), and routes flagged transactions to a human approver in a Next.js dashboard. Policies and the approver set live on-chain, owned by an authority that can be a multisig or DAO.*
>
> *Here is a brainstormed list of all potential user types: [pasted the 12 from B.2 verbatim].*
>
> *Based on the value proposition, which 2–5 of these user types are the most critical to focus on for an initial Proof-of-Concept? For each user you recommend, provide a brief rationale.*

Synthesized AI output:

The AI recommended five: agent integrator, agent signer, approver, policy authority, and compliance officer. Its rationales for the first four were essentially what I'd written myself. Its rationale for the compliance officer was "they validate the audit log property of the system, which is a core value area."

My take and final decision:

I kept four of the five and dropped compliance officer. The compliance officer reads outputs that the other four produce; they don't drive the loop. Including them as a POC persona would add user stories (S-something: compliance officer queries the audit log for a date range) that depend on every other persona's stories already working. Including them inflates scope without proving anything new.

I also collapsed "policy author" and "policy authority" from my brainstorm into a single persona (P4). They are functionally the same on day 1, and the distinction (someone who *drafts* policy text vs. someone who *signs the on-chain mutation*) only matters when there's a policy review process between drafting and signing — which is post-POC.

Final four personas: P1 integrator, P2 agent signer, P3 approver, P4 policy authority. See deliverable §2.

Where I disagreed with the AI: the compliance officer call. It's not that the persona is wrong — it's that they're a *consumer* of POC outputs, not a *driver* of them, and the POC is scope-limited to what proves the loop.

Where I agreed: the disambiguation between integrator (writes code) and agent signer (runs code). The AI's framing here was tighter than my brainstorm draft and I kept it.

B.4 — Function mapping prompt (Part A.3)

My prompt:

> *For Cordon (value prop above) and focusing on these four prioritized user types — integrator, agent signer, approver, policy authority — help map out the key functions or interactions each user would need to perform. Keep functions concrete and single-action; avoid wrapper verbs like "manage" or "configure."*

Synthesized AI output:

A function map roughly matching the table in deliverable §3, with these differences:

- The AI gave the integrator a function "monitors agent activity" — I dropped it. "Monitors" is a category, not an action. I broke the underlying need into F1.4 (read pending queue) and F1.5 (read audit log), which are concrete.
- The AI gave the agent signer a single function "submits transactions and handles responses." I split it into F2.1–F2.6 because the response handling has three distinct branches (auto-approved, queued, denied) and the queued branch has a second event (approver decision) that arrives later.
- The AI did not include the kill switch (F4.6) or the authority transfer (F4.7). I added both manually. The kill switch is the single most important governance function — without it the "operator can't quietly mutate policy" value claim doesn't hold. The authority transfer is the mechanism that makes "policy authority can be a multisig or DAO" a real property, not just a slogan.

- The AI grouped “add approver” and “remove approver” into one function. I split them (F4.4 / F4.5) because they’re two distinct on-chain instructions with distinct gating concerns.

The AI’s wrapper-verb tendency is a real anti-pattern for this exercise. Functions like “manages policies” are useless input for translating into on-chain instructions because they don’t correspond to a single instruction call.

B.5 — Top critical interactions and initial technical requirements (Part A.4)

I identified the two paths in deliverable §4 manually first (auto-approval loop, HITL loop), then prompted the AI for technical requirements.

My prompt:

> *Based on these two critical user interactions for Cordon — (Path A) agent submits intent, policy auto-approves under cap, transaction broadcasts; (Path B) agent submits intent, policy flags it, approver decides — what are the key technical requirements to build a POC on Solana / Anchor?*

Synthesized AI output:

- An Anchor program with instructions for registering an agent, submitting an intent, checking the intent against policy, approving, and rejecting.
- A `Policy` account per agent storing the rules and the approver set.
- A `PendingApproval` (the AI’s name; I renamed to `Intent`) account per submitted transaction.
- An off-chain SDK that wraps the agent’s signing path through these instructions.
- A dashboard that queries pending intents via `getProgramAccounts` with a memcmp filter.
- An on-chain event stream for the audit log.

My take:

This is correct and roughly what I’d already sketched in the prior Cordon repo. The one substantive edit was renaming `PendingApproval` → `Intent` and making the account hold the *full intent lifecycle*, not just the pending state. The reason: if “pending” and “approved” are two different account types, then the audit log has to be the union of two queries with two different account layouts. If `Intent` has a status enum that progresses (`Submitted → AutoApproved | Pending → HumanApproved | Rejected | Expired`), the audit log is one query against one account type, filterable by status. Cheaper to index, simpler to reason about.

The AI also missed the kill-switch effect on `set_spend_cap` (a paused policy shouldn’t be tunable, otherwise a compromised authority can pause-then-loosen-then-resume). I added that constraint and propagated it into S4 / S21 in the deliverable.

B.6 — Adversarial critique (Part B)

My prompt:

> *Review my core user functions / stories (deliverable §3 and §5 draft) and requirements (§6 draft). Considering Cordon's value prop, do these stories truly hit the mark? Are the requirements granular enough to map to specific Anchor instructions, account types, and PDA seeds? What's missing or unclear?*

Synthesized critique:

1. **"Agent" is ambiguous.** The stories sometimes say "agent" when they mean the integrator and sometimes when they mean the runtime signer. Force a disambiguation across every story.
2. **No story for the expiry case.** What happens if an intent sits in the queue past the TTL and no approver acts? The stories cover approve and reject but not expire.
3. **No story for the authority-transfer.** The persona is defined but the function isn't reified as a story.
4. **The audit log is implied, not stated.** "Cordon records the intent on-chain" appears nowhere as a numbered story; it's only implicit in S8.
5. **Setup stories conflate steps.** "Integrator installs Cordon and configures their agent" is three actions: SDK install, keypair generation, policy creation.
6. **No read stories.** The deliverable has write stories (mutating on-chain state) but no read stories (querying it). The dashboard and the SDK both need read paths.
7. **Rate limiting is hand-waved.** "Under the rate limit" is in the policy check but isn't grounded — what's the window, where is the counter stored?

My response, item by item:

1. **Valid.** Renamed every actor across the deliverable. "Agent integrator" → off-chain code-writer (P1). "Agent signer" → runtime programmatic actor (P2). Fixed every story to name one or the other, never just "agent."
2. **Valid.** Added S20 (drop on rejection or expiry) and added explicit `Intent.status == Expired` handling in S17 and the optional `expire_intent` instruction in S20's requirements.
3. **Valid.** Added S22 (transfer authority) plus the two-step propose / accept mechanism.
4. **Partially valid.** I added S9 ("Cordon records the intent on-chain so the decision is auditable") explicitly as a story for legibility, but in the on-chain requirements I noted that S9 is *covered* by S8 — the `Intent` account IS the on-chain record. No separate log account is needed. The reviewer's instinct was right that the property needed to be named; the implementation collapses into one account type, not two.
5. **Valid.** Setup stories S1, S2, S3 in the deliverable are the split-out form. Original draft had a single "integrator sets up agent" story; refinement log entry C1 below has the before / after.
6. **Valid.** Added S23 (read pending queue) and S24 (read decision log) as explicit stories and grounded their account-layout requirements (memcmp-friendly status byte and agent_signer pubkey at fixed offsets).
7. **Valid but deferred.** Added "rate-limit window representation" as open question #3 in deliverable §7. For the requirements stage it's enough to say "the policy check evaluates the rate limit"; choosing sliding-vs-fixed-vs-token-bucket is an architecture decision.

The critique I *rejected*: an early draft of the AI critique also suggested adding stories for "approver requests more information from the agent before deciding." That's a real product feature but it isn't a POC requirement — for the POC the approver makes a binary decision on the simulation output

already on the `Intent`. Adding a request-info loop would require an off-chain channel (Slack, email) the POC doesn't have. Flagged for post-POC.

B.7 — Part C refinement log (granularity, atomicity, de-jargon)

Every change to a user story between the post-critique draft and the final §5 list, with before / after / rationale.

C1 — Atomicity split

- **Before:** "As an integrator, I install Cordon and set up my agent's policy."
- **After:** S1 (install SDK) + S2 (generate keypair) + S3 (create policy account) + S4 (set spend cap) + S5 (add allowlist entry) + S6 (add approver).
- **Rationale:** Original was six distinct actions in one sentence. Each maps to a different on-chain instruction (S3 → `register_agent`, S4 → `set_spend_cap`, etc.) or to no on-chain action at all (S1, S2 are off-chain). Splitting lets each on-chain requirement attach to one story.

C2 — Atomicity split on the runtime path

- **Before:** "As an agent, I submit a transaction and Cordon checks it and either it goes through or it gets queued."
- **After:** S7 (build tx) + S8 (submit intent) + S9 (Cordon records intent) + S10 (Cordon checks against policy) + S11 (mark auto-approved) + S12 (broadcast).
- **Rationale:** Original collapsed four distinct on-chain state transitions and one off-chain broadcast. The dashboard's audit log query depends on each transition being its own story so that "what happened to intent X" can be read off the `Intent.status` history.

C3 — Actor disambiguation

- **Before:** "As an agent, I submit my transaction to Cordon."
- **After:** "As an agent signer, I submit that transaction to Cordon for a policy check." (S8)
- **Rationale:** "Agent" was ambiguous (see B.6 #1). The runtime signer is the actor; the integrator is the configurer.

C4 — De-jargon

- **Before:** "As Cordon, I invoke the `check_intent` CPI against the policy program and mutate `Intent.status` based on the result."
- **After:** "As Cordon, I check the intent against the policy account's rules." (S10)
- **Rationale:** "CPI" and "mutate state" are implementation language. The story should describe the action; the on-chain requirements (deliverable §6 under S10) can name the instruction and the state transition.

C5 — De-jargon

- **Before:** “As an approver, I see the pending intents that match my pubkey in the approver set via a `getProgramAccounts` query with a memcmp filter.”
- **After:** “As an approver, I open the dashboard and see every pending intent on agents I’m listed on.” (S15)
- **Rationale:** The user story describes the user’s experience; the requirements document describes the query mechanism. Kept the `getProgramAccounts` / memcmp detail in §6 under S15 where it belongs.

C6 — Atomicity split on approver action

- **Before:** “As an approver, I review a pending intent and either approve or reject it.”
- **After:** S16 (open and read details) + S17 (approve) + S18 (reject).
- **Rationale:** Approve and reject are different on-chain instructions with different post-conditions. Opening the intent to read it is a third distinct action (no on-chain write). Three stories, three different requirements.

C7 — Missing story added

- **Before:** [no story]
- **After:** S20 (“As an agent signer, I drop the transaction when an approver rejects it or the pending intent expires.”)
- **Rationale:** The rejection and expiry paths were implicit in the design but absent from the story list. Without the story, the on-chain `expire_intent` requirement had nowhere to attach.

C8 — Missing story added

- **Before:** [no story]
- **After:** S21 (kill switch), S22 (transfer authority)
- **Rationale:** P4’s most important functions (F4.6, F4.7) had no corresponding stories. Without stories, no on-chain requirements would have been brainstormed in Part D. Added.

C9 — Read stories added

- **Before:** [no stories — only write paths]
- **After:** S23 (read pending queue), S24 (read decision log)
- **Rationale:** Critique #6. The on-chain account layout has read-side requirements (memcmp-friendly offsets) that only emerge if you write the read stories down.

C10 — Redundancy elimination

- **Before:** Earlier draft had both “As Cordon, I write the intent to the audit log” and “As Cordon, I record the intent on-chain so the decision is auditable.”
- **After:** Single story S9.
- **Rationale:** Same action described twice. The audit log *is* the on-chain record; there is no separate log account (see B.6 #4).

C11 — De-jargon

- **Before:** "As a policy authority, I rotate the kill switch on the policy registry."
- **After:** "As a policy authority, I flip the kill switch to pause every new intent for my agent." (S21)
- **Rationale:** "Rotate" was the wrong verb (rotation implies key rotation). "Flip" is plain. Added the *consequence* ("pause every new intent for my agent") so a non-technical reader understands what the action does.

C12 — De-jargon

- **Before:** "As a policy authority, I transfer ownership of the `Policy` PDA to a new authority pubkey."
- **After:** "As a policy authority, I transfer policy authority to a new wallet." (S22)
- **Rationale:** "PDA" and "pubkey" are Solana-specific. "Policy authority" and "wallet" are the same idea in user-facing language.

B.8 — Part D approach notes

The deliverable §6 brainstorm follows the assignment's prescribed pattern (one story → bulleted on-chain requirements), but two patterns are worth flagging because they shaped a lot of the bullets:

1. "No on-chain requirements" is a valid bullet. Several stories (S1, S2, S7, S12, S15, S16, S19, S20, S23, S24) are off-chain or read-only. I marked them explicitly rather than inventing on-chain machinery to fill space. A story that doesn't need an instruction shouldn't get one.

2. Several stories collapse into one instruction with state transitions. S8 + S9 + S10 + S11 all describe the lifecycle of one `Intent` account. The requirements list reflects that — `submit_intent` covers S8 and S9; `check_intent` (or the folded version inside `submit_intent`) covers S10 and S11. I noted the open question in §7 (#1) rather than pretending the architecture decision was already made.

3. Validation and gating are first-class requirements, not afterthoughts. Every mutation instruction has a gating bullet (who can sign) and at least one failure bullet (when it must fail). This is the granularity needed to map to Anchor account constraints (`has_one`, `constraint = ...`) without ambiguity in the next assignment.

Appendix — Sources

- Capstone Assignment 1 (Cordon proposal): `cordon-proposal.pdf` at repo root.
- Cordon code-in-progress: https://github.com/Hijanhv/cordon
- Anchor framework reference (PDA seeds, account constraints, instruction signatures): https://www.anchor-lang.com/
- Solana `getProgramAccounts` + memcmp filter docs: https://solana.com/docs/rpc/http/getprogramaccounts